

HARVARD UNIVERSITY

School of Engineering and Applied Sciences

TP — Smart Inventory & Order Platform

Full-Stack Development with
Spring Boot 3.x & **Angular 18+**

Course: CS-E4200 — Modern Full-Stack Web Engineering

Professor: Dr. Y. Elkhamlichi

Semester: Fall 2026

Duration: 6 Weeks (2h/day)

Pedagogical Approach — The 70/30 Golden Rule

30% reading documentation & watching concepts



70% hands-on coding inside a single repository

Every concept is immediately applied in the project. No isolated toy examples.

Contents

1	What You Will Learn	3
1.1	Spring Boot 3.x (Backend)	3
1.2	Angular 18+ (Frontend)	3
1.3	Architecture & DevOps	4
2	Project Overview & Architecture	5
2.1	Smart Inventory & Order Platform	5
2.2	Backend Architecture (Spring Boot)	5
2.3	Frontend Architecture (Angular)	6
3	Phase 1 — Data Contract & Basic Rendering	8
3.1	Exercise 1.1 — Initialize the Spring Boot Project	8
3.2	Exercise 1.2 — Design the Data Model	9
3.3	Exercise 1.3 — Build the Repository Layer	9
3.4	Exercise 1.4 — Create the DTO Layer	9
3.5	Exercise 1.5 — Implement the Service Layer	10
3.6	Exercise 1.6 — Build the REST Controller	10
3.7	Exercise 1.7 — Configure CORS	11
3.8	Exercise 1.8 — Initialize the Angular Project	11
3.9	Exercise 1.9 — Create TypeScript Models	11
3.10	Exercise 1.10 — Build the Product Service	12
3.11	Exercise 1.11 — Create the Product List Component	12
3.12	Exercise 1.12 — Navigation & Layout	13
4	Phase 2 — Reactive Forms & Error Handling	14
4.1	Exercise 2.1 — Backend Validation	14
4.2	Exercise 2.2 — Global Error Handler	14
4.3	Exercise 2.3 — Product Form (Angular)	15
4.4	Exercise 2.4 — Display Server-Side Errors	15
4.5	Exercise 2.5 — Custom Validator	15
5	Phase 3 — Signals & RxJS Pipeline	16
5.1	Exercise 3.1 — Backend Pagination & Sorting	16
5.2	Exercise 3.2 — Dynamic Search with JPA Specifications	16
5.3	Exercise 3.3 — Angular Signals	17
5.4	Exercise 3.4 — Live Search with RxJS	17
5.5	Exercise 3.5 — Reusable Pagination Component	17

6	Phase 4 — JWT Security & Interceptors	19
6.1	Exercise 4.1 — User Entity & Auth DTOs	19
6.2	Exercise 4.2 — JWT Service	19
6.3	Exercise 4.3 — Spring Security Configuration	20
6.4	Exercise 4.4 — Auth Service & Controller	20
6.5	Exercise 4.5 — Angular Auth & Interceptors	20
6.6	Exercise 4.7 — Route Guards & Auth Pages	21
7	Phase 5 — Production Readiness	22
7.1	Exercise 5.1 — N+1 Problem	22
7.2	Exercise 5.2 — Angular Deferred Loading	22
7.3	Exercise 5.3 — Docker Compose	22
7.4	Exercise 5.4 — Admin Panel	23
I	Solutions	24
8	Solution — Phase 1	25
8.1	Solution 1.1 — application.yml	25
8.2	Solution 1.2 — Data Model	25
8.3	Solution 1.3 — Repositories	27
8.4	Solution 1.4 — DTOs	27
8.5	Solution 1.5 — Service Layer	28
8.6	Solution 1.6 — Controller	30
8.7	Solution 1.7 — CORS	31
8.8	Solution 1.8–1.12 — Angular Setup	31
9	Solution — Phase 2	35
9.1	Solution 2.1 — Bean Validation	35
9.2	Solution 2.2 — Global Error Handler	36
9.3	Solution 2.3–2.5 — Angular Forms	37
10	Solution — Phase 3	40
10.1	Solution 3.1–3.2 — Pagination & Specifications	40
10.2	Solution 3.3–3.4 — Signals & RxJS	41
11	Solution — Phase 4	44
11.1	Solution 4.1 — User Entity	44
11.2	Solution 4.2–4.3 — JWT & Security Config	45
11.3	Solution 4.5–4.7 — Angular Auth	45
12	Solution — Phase 5	47
12.1	Solution 5.1 — N+1 Fix	47
12.2	Solution 5.3 — Docker	47
13	Weekly Time Distribution	49
13.1	Daily Schedule (2 hours/day)	49
14	Essential Resources	50

Chapter 1

What You Will Learn

By the end of this TP, you will have built a **complete, production-grade** full-stack application and mastered the following concepts:

1.1 Spring Boot 3.x (Backend)

Concept	Description
Layered Architecture	Controller → Service → Repository — strict separation of concerns
JPA / Hibernate	Entity mapping, relationships, <code>@EntityGraph</code> , DTO Projections
REST API Design	Versioned endpoints, proper HTTP verbs, status codes
Bean Validation	<code>@Valid</code> , <code>@NotBlank</code> , <code>@Min</code> , custom constraints
Global Error Handling	<code>@RestControllerAdvice</code> , <code>@ExceptionHandler</code> , structured error maps
Pagination & Sorting	<code>Pageable</code> , <code>Page<T></code> , dynamic search with JPA Specifications
Spring Security + JWT	Stateless authentication, <code>JwtAuthenticationFilter</code> , role-based access
Performance	N+1 problem diagnosis, <code>@EntityGraph</code> , DTO Projections

1.2 Angular 18+ (Frontend)

Concept	Description
Standalone Architecture	No <code>NgModule</code> — <code>provideHttpClient()</code> , <code>provideRouter()</code>
Strongly-Typed Services	<code>HttpClient</code> with TypeScript interfaces

Concept	Description
Modern Control Flow	<code>@for</code> , <code>@if</code> , <code>@switch</code> , <code>@defer</code>
Reactive Forms	<code>FormGroup</code> , <code>FormControl</code> , <code>Validators</code> , custom validators
Angular Signals	<code>signal()</code> , <code>computed()</code> , <code>effect()</code> for reactive state
RxJS Operators	<code>debounceTime</code> , <code>distinctUntilChanged</code> , <code>switchMap</code>
HTTP Interceptors	<code>HttpInterceptorFn</code> for token injection and error handling
Route Guards	<code>CanActivateFn</code> for protecting routes
Lazy Loading	<code>@defer</code> (on viewport) for performance optimization

1.3 Architecture & DevOps

Concept	Description
Clean Project Structure	Industry-standard folder naming used by top tech companies
Full-Stack Integration	Frontend ↔ Backend communication with proper CORS
Docker Compose	Multi-container deployment (DB + API + UI)
Debugging Skills	Network Tab, SQL logs, error tracing

Chapter 2

Project Overview & Architecture

2.1 Smart Inventory & Order Platform

You are building a **professional inventory management system** with three main modules:

-  **Products Module:** Live search, category filtering, pagination, sorting
-  **Cart Module:** Add/remove items, update quantities, server sync
-  **Admin Panel:** Role-based access, price management, stock control

2.2 Backend Architecture (Spring Boot)

Spring Boot Project Structure

```
smart-inventory-api/  
|-- src/main/java/com/smartinventory/  
|   |-- SmartInventoryApplication.java  
|   |-- config/  
|       |-- CorsConfig.java  
|       |-- SecurityConfig.java  
|   |-- controller/  
|       |-- AuthController.java  
|       |-- ProductController.java  
|       |-- CategoryController.java  
|       |-- CartController.java  
|       |-- AdminController.java  
|   |-- dto/  
|       |-- request/  
|           |-- LoginRequest.java  
|           |-- RegisterRequest.java  
|           |-- ProductRequest.java  
|           |-- CartItemRequest.java  
|       |-- response/  
|           |-- AuthResponse.java  
|           |-- ProductResponse.java
```

```

| | | |-- PageResponse.java
| | | |-- ErrorResponse.java
| | |-- exception/
| | | |-- GlobalExceptionHandler.java
| | | |-- ResourceNotFoundException.java
| | | |-- BusinessException.java
| | |-- model/
| | | |-- Product.java
| | | |-- Category.java
| | | |-- CartItem.java
| | | |-- Order.java
| | | |-- User.java
| | |-- repository/
| | | |-- ProductRepository.java
| | | |-- CategoryRepository.java
| | | |-- CartItemRepository.java
| | | |-- UserRepository.java
| | |-- security/
| | | |-- JwtService.java
| | | |-- JwtAuthenticationFilter.java
| | | |-- UserDetailsServiceImpl.java
| |-- service/
| | |-- ProductService.java
| | |-- CategoryService.java
| | |-- CartService.java
| | |-- AuthService.java
| | |-- impl/
| | | |-- ProductServiceImpl.java
| | | |-- CategoryServiceImpl.java
| | | |-- CartServiceImpl.java
| | | |-- AuthServiceImpl.java
+-- pom.xml

```

2.3 Frontend Architecture (Angular)

Angular Project Structure

```

smart-inventory-ui/
+-- src/app/
| |-- core/
| | |-- interceptors/
| | | |-- auth.interceptor.ts
| | | |-- error.interceptor.ts
| | |-- guards/
| | | |-- auth.guard.ts
| | | |-- admin.guard.ts
| |-- services/
| | |-- auth.service.ts
| | |-- storage.service.ts

```

```
|-- features/  
|  |-- auth/  
|    |-- login/  
|    |-- register/  
|    |-- models/  
|    +-- auth.routes.ts  
|  |-- products/  
|    |-- product-list/  
|    |-- product-form/  
|    |-- models/  
|    |-- services/  
|    +-- products.routes.ts  
|  |-- cart/  
|  +-- admin/  
|-- shared/  
|  |-- components/  
|    |-- navbar/  
|    +-- pagination/  
|  +-- pipes/  
|-- app.component.ts  
|-- app.config.ts  
+-- app.routes.ts
```

Why?

Why this structure? This is the standard used at companies like Google, Apple, and Microsoft.

- **core/** — Singleton services, guards, interceptors (instantiated ONCE)
- **features/** — Domain modules, each self-contained
- **shared/** — Reusable components, pipes, directives

Chapter 3

Phase 1 — Data Contract & Basic Rendering

Objective: Establish the full-stack connection. By the end of this phase, your Angular app will fetch and display a list of products from your Spring Boot API.

Estimated Time: 4 days (2h/day)

3.1 Exercise 1.1 — Initialize the Spring Boot Project

Context: Every professional project starts with a well-configured foundation.

Tasks:

1. Go to start.spring.io and generate a project with:
 - Group: `com.smartinventory`, Artifact: `smart-inventory-api`
 - Java 21
 - Dependencies: Spring Web, Spring Data JPA, PostgreSQL Driver, Lombok, Validation
2. Configure your database connection in `application.yml`. Think about:
 - What is the JDBC URL format for PostgreSQL?
 - What property controls table auto-creation in JPA?
 - How do you enable SQL logging?
3. Create ALL the package folders shown in the Architecture Blueprint.

Think About It

Why do we separate `dto/request` and `dto/response` into sub-packages? What problem does this solve compared to putting all DTOs in a single package?

Checkpoint

Your Spring Boot application starts without errors, connects to PostgreSQL, and the console shows the Hibernate dialect being used.

3.2 Exercise 1.2 — Design the Data Model

Context: The `model/` package contains your JPA entities. Start with `Product` and `Category`.

Tasks:

1. Create the `Category` entity: `id` (Long, auto-generated), `name` (String, unique), `description` (String), and a one-to-many relationship with products.
2. Create the `Product` entity: `id`, `name`, `description`, `price` (BigDecimal), `stockQuantity` (Integer), `imageUrl`, `createdAt`, `updatedAt`, and a many-to-one relationship with category.

Think About It

- Why do we use `BigDecimal` for price instead of `Double`?
- What JPA annotation makes `createdAt` auto-populate on first save?
- What is the difference between `FetchType.LAZY` and `FetchType.EAGER`?

Hints

Look at `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@CreationTimestamp`, `@UpdateTimestamp`, `@ManyToOne`, `@OneToMany(mappedBy = "...")`

Checkpoint

Restart the app. Hibernate should auto-create `products` and `categories` tables.

3.3 Exercise 1.3 — Build the Repository Layer

Context: In Spring Data JPA, you don't write SQL — you extend an interface.

Tasks:

1. Create `CategoryRepository` extending the appropriate Spring Data interface.
2. Create `ProductRepository` with a custom derived query method to find products by category ID.

Think About It

Spring Data JPA uses “Derived Query Methods.” If your entity has a field `category` with a sub-field `id`, what method name would automatically generate `WHERE category_id = ??`

3.4 Exercise 1.4 — Create the DTO Layer

Context: Never expose your entities directly in API responses. DTOs act as a shield between your database model and the outside world.

Tasks:

1. Create `ProductResponse` record with: `id`, `name`, `price`, `stockQuantity`, `imageUrl`, `categoryName`
2. Create `CategoryResponse` record with `id` and `name`
3. Create `ProductRequest` record with all creation fields including `categoryId`

Think About It

- Why do we use Java `record` instead of a regular class for DTOs?
- Why does `ProductResponse` contain `categoryName` instead of `categoryId`?
- What would happen if we returned the entity directly with a lazily loaded field?

3.5 Exercise 1.5 — Implement the Service Layer

Context: The Service layer contains business logic. It receives DTOs from controllers and returns DTOs back.

Tasks:

1. Create a `ProductService` **interface** with: `getAllProducts()`, `getProductById(Long id)`, `createProduct(ProductRequest request)`
2. Create `ProductServiceImpl` implementing the interface:
 - Inject repositories via constructor injection
 - Convert entities to DTOs in each method
 - Throw `ResourceNotFoundException` when not found

Think About It

- Why do we define an interface separate from the implementation?
- Why constructor injection instead of `@Autowired` on fields?
- Should entity-to-DTO conversion live in the service or a separate mapper?

3.6 Exercise 1.6 — Build the REST Controller

Context: Controllers are the entry point of your API. They should be **thin** — no business logic.

Tasks:

1. Create `ProductController` mapped to `/api/v1/products`
2. Implement: `GET /` (list), `GET /{id}` (detail), `POST /` (create with HTTP 201)

 Think About It

- Why do we version the API with `/api/v1/`?
- Should the controller return `ResponseEntity<T>` or just `T`?

 Checkpoint

Test with Postman: create a product via POST, retrieve all via GET.

3.7 Exercise 1.7 — Configure CORS

Context: Your Angular app runs on port 4200, your API on 8080. Browsers block cross-origin requests by default.

Tasks:

1. Create `CorsConfig` implementing `WebMvcConfigurer`
2. Allow requests from `http://localhost:4200`
3. Allow methods: GET, POST, PUT, DELETE, OPTIONS

 Think About It

Why is CORS a server-side configuration? In production, should you allow `*` (all origins)?

3.8 Exercise 1.8 — Initialize the Angular Project

Tasks:

1. Generate: `ng new smart-inventory-ui -standalone -style=css -routing -ssr=false`
2. Create ALL folders from the Angular Architecture Blueprint
3. Configure environments with `apiUrl`
4. Set up `app.config.ts` with `provideHttpClient(withFetch())` and `provideRouter(routes)`

 Think About It

What is the difference between the old `NgModule` architecture and `Standalone`? Why `withFetch()`?

3.9 Exercise 1.9 — Create TypeScript Models

Tasks:

1. In `features/products/models/product.model.ts`, create `Product` and `ProductRequest` interfaces

2. Ensure they mirror the backend DTOs exactly

Think About It

Why do we use `interface` and not `class`? What happens if the backend adds a field you don't have?

3.10 Exercise 1.10 — Build the Product Service

Tasks:

1. Create `product.service.ts` with `@Injectable({ providedIn: 'root' })`
2. Implement strongly-typed methods: `getAll()`, `getById(id)`, `create(request)`

Think About It

Why does `HttpClient.get<Product[]>()` return an `Observable` instead of a `Promise`?

3.11 Exercise 1.11 — Create the Product List Component

Tasks:

1. Generate: `ng generate component features/products/product-list --standalone`
2. Inject `ProductService`, fetch data in `ngOnInit()`
3. Use `@for` (not `*ngFor`) with `track` to display products
4. Use `@if` to show “Out of Stock” conditionally
5. Use `@empty` block for empty lists
6. Set up routing in `app.routes.ts`

Think About It

What is the `track` expression in `@for`? What is the memory leak risk with `subscribe()` in `ngOnInit()`?

Checkpoint

Navigate to `http://localhost:4200/products`. Products from your API should appear.

3.12 Exercise 1.12 — Navigation & Layout

Tasks:

1. Create `NavbarComponent` in `shared/components/navbar/`
2. Use `routerLink` (not `href`) and `routerLinkActive`
3. Update `app.component.html` with navbar and `<router-outlet>`



Think About It

Why `routerLink` instead of `href`? What happens to the page state with `href`?

Chapter 4

Phase 2 — Reactive Forms & Error Handling

Objective: Build a product creation form with client-side and server-side validation.

4.1 Exercise 2.1 — Backend Validation

Tasks:

1. Add validation annotations to `ProductRequest`: `@NotBlank`, `@Size`, `@DecimalMin`, `@Min`, `@NotNull`
2. Activate validation with `@Valid` on the controller parameter

Think About It

What is the difference between `@NotNull`, `@NotEmpty`, and `@NotBlank`?

4.2 Exercise 2.2 — Global Error Handler

Tasks:

1. Create `ErrorResponse` record: `timestamp`, `status`, `error`, `errors` (Map)
2. Create `GlobalExceptionHandler` with `@RestControllerAdvice`:
 - Handle `MethodArgumentNotValidException` → HTTP 400 with field errors
 - Handle `ResourceNotFoundException` → HTTP 404
 - Handle generic `Exception` → HTTP 500 (never expose stack traces!)

Think About It

Why is `@RestControllerAdvice` better than try-catch in every controller? What's the difference between what you *log* and what you *return*?

4.3 Exercise 2.3 — Product Form (Angular)

Tasks:

1. Build a `FormGroup` with `FormControl` and `Validators` for each field
2. On submit: `markAllAsTouched()`, `validate`, then call the service
3. Display validation errors below each field (only when touched AND invalid)

Think About It

What is “touched” vs “dirty”? Why `markAllAsTouched()` on submit?

4.4 Exercise 2.4 — Display Server-Side Errors

Tasks:

1. Create a `serverErrors` object to store error map from HTTP 400 responses
2. Display server errors under the corresponding fields
3. Clear server errors when the user starts typing (listen to `valueChanges`)

4.5 Exercise 2.5 — Custom Validator

Tasks:

1. Create a custom validator function for price format (max 2 decimal places)
2. Return `{ invalidPrice: true }` if invalid, `null` if valid
3. Apply to the price `FormControl` and display the error message

Think About It

What is the difference between a synchronous and async validator? When would you need async?

Chapter 5

Phase 3 — Signals & RxJS Pipeline

Objective: Implement live search, pagination, and sorting. Master Angular Signals and RxJS operators.

5.1 Exercise 3.1 — Backend Pagination & Sorting

Tasks:

1. Modify service to accept `Pageable` and return `Page<ProductResponse>`
2. Create `PageResponse<T>` generic DTO
3. Update controller to accept `page`, `size`, `sort` query parameters

Think About It

What SQL query does `Pageable` generate? What is the difference between `Slice<T>` and `Page<T>`?

5.2 Exercise 3.2 — Dynamic Search with JPA Specifications

Tasks:

1. Extend `ProductRepository` with `JpaSpecificationExecutor<Product>`
2. Create static specification methods: `hasName()`, `hasCategory()`, `hasPriceRange()`
3. Combine specifications with `.and()` in the service layer

Think About It

How does the Specification pattern implement the Strategy design pattern? What if all parameters are null?

5.3 Exercise 3.3 — Angular Signals

Tasks:

1. Replace plain variables with `signal()`: `products`, `loading`, `currentPage`
2. Create `computed()` signals for derived state
3. Update templates to use `()` syntax: `products()`, `loading()`
4. Use `.set()` and `.update()` appropriately

Think About It

What is the difference between `signal.set()` and `signal.update()`? How do Signals compare to React's `useState()`?

5.4 Exercise 3.4 — Live Search with RxJS

Tasks:

1. Add a search `FormControl` bound to an input
2. Build the RxJS pipeline: `valueChanges` $\rightarrow$ `debounceTime(300)` $\rightarrow$ `distinctUntilChanged()` $\rightarrow$ `switchMap()` $\rightarrow$ `subscribe()`
3. Clean up with `takeUntil(destroy$)` in `ngOnDestroy()`

Think About It

Why is `switchMap` the correct operator, not `mergeMap` or `concatMap`? What does “cancel the previous request” mean at the HTTP level?

Checkpoint

Type in the search box. Check the Network tab — previous requests should be cancelled.

5.5 Exercise 3.5 — Reusable Pagination Component

Tasks:

1. Create `PaginationComponent` in `shared/components/pagination/`
2. Define inputs: `currentPage`, `totalPages`
3. Define output: `pageChange` event
4. Display Previous/Next buttons with page numbers



Think About It

What is a “smart” vs “dumb/presentational” component? Which is `PaginationComponent`?

Chapter 6

Phase 4 — JWT Security & Interceptors

Objective: Implement complete authentication and authorization.

6.1 Exercise 4.1 — User Entity & Auth DTOs

Tasks:

1. Create `User` entity with: `id`, `firstName`, `lastName`, `email`, `password` (hashed), `role` (enum: `USER`, `ADMIN`)
2. Create `LoginRequest`, `RegisterRequest`, `AuthResponse` DTOs
3. Create `UserRepository` with `findByEmail()` and `existsByEmail()`

Think About It

Why hashed passwords? Should `Role` be an entity or enum? Where should password-match validation happen?

6.2 Exercise 4.2 — JWT Service

Tasks:

1. Add `jjwt` dependencies to `pom.xml`
2. Implement `JwtService`: `generateToken()`, `extractUsername()`, `isTokenValid()`
3. Store secret key in `application.yml`

Think About It

What are the three parts of a JWT? Why is the secret key critical? Symmetric vs asymmetric signing?

6.3 Exercise 4.3 — Spring Security Configuration

Tasks:

1. Create `UserDetailsServiceImpl` implementing `UserDetailsService`
2. Create `JwtAuthenticationFilter` extending `OncePerRequestFilter`
3. Create `SecurityConfig`: disable CSRF, set STATELESS sessions, configure URL authorization, add JWT filter

Think About It

Why disable CSRF for a REST API? What does “stateless” mean? Why must the JWT filter run before `UsernamePasswordAuthenticationFilter`?

6.4 Exercise 4.4 — Auth Service & Controller

Tasks:

1. Implement `AuthService`: register (check duplicates, hash password, generate token), login (authenticate, generate token)
2. Create `AuthController`: POST `/auth/register`, POST `/auth/login`

Checkpoint

Register → get token. Login → get token. Use token to access GET `/products`.

6.5 Exercise 4.5 — Angular Auth & Interceptors

Tasks:

1. Create `StorageService`: `setToken()`, `getToken()`, `removeToken()`, `isLoggedIn()`
2. Create `AuthService`: `login()`, `register()`, `logout()`, `getUserRole()`
3. Create `authInterceptor`: clone requests and add `Authorization: Bearer <token>`
4. Create `errorInterceptor`: redirect on 401, handle 403
5. Register interceptors in `app.config.ts`

Think About It

Why clone the request? Why skip auth endpoints? Why is `localStorage` a security risk (XSS)?

6.6 Exercise 4.7 — Route Guards & Auth Pages

Tasks:

1. Create `authGuard (CanActivateFn)`: check login, redirect to `/auth/login`
2. Create `adminGuard`: check ADMIN role
3. Apply guards to routes
4. Build `LoginComponent` and `RegisterComponent` with Reactive Forms
5. Implement cross-field validator for password confirmation

✓ Checkpoint

Complete auth flow: Register → Login → Access protected pages → Logout → Redirect.

Chapter 7

Phase 5 — Production Readiness

Objective: Optimize performance, solve N+1, add lazy loading, containerize with Docker.

7.1 Exercise 5.1 — N+1 Problem

Tasks:

1. Enable SQL logging, count queries for a product list
2. Fix with `@EntityGraph(attributePaths = {"category"})`
3. Alternative: DTO Projection with JPQL `SELECT new`



Think About It

Why is `FetchType.EAGER` not a good fix? When is DTO Projection better than `@EntityGraph`?

7.2 Exercise 5.2 — Angular Deferred Loading

Tasks:

1. Wrap heavy components in `@defer (on viewport)` blocks
2. Create skeleton loader CSS animations
3. Experiment with `(on idle)`, `(on timer)`, `(when condition)`

7.3 Exercise 5.3 — Docker Compose

Tasks:

1. Create multi-stage `Dockerfile` for Spring Boot (Maven build → JRE run)
2. Create multi-stage `Dockerfile` for Angular (Node build → Nginx serve)
3. Create `nginx.conf` with API proxy and SPA routing

4. Create `docker-compose.yml` with 3 services: db, api, ui

Think About It

Why multi-stage builds? Why does Nginx proxy API requests? What is a Docker volume?

Checkpoint

`docker-compose up -build` → Full application at `http://localhost`.

7.4 Exercise 5.4 — Admin Panel

Tasks:

1. Backend: `AdminController` with PUT/DELETE endpoints, protected by `@PreAuthorize`
2. Frontend: Dashboard with editable product table, delete with confirmation
3. Route protection with `adminGuard`

Think About It

Why do we need BOTH `@PreAuthorize` (backend) AND route guards (frontend)? What if you only protect the frontend?

Part I

Solutions

Chapter 8

Solution — Phase 1

8.1 Solution 1.1 — application.yml

application.yml

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/smart_inventory
    username: postgres
    password: secret
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

8.2 Solution 1.2 — Data Model

model/Category.java

```
1 package com.smartinventory.model;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import java.util.List;
6
7 @Entity
8 @Table(name = "categories")
```

```
9  @Getter @Setter
10 @NoArgsConstructor @AllArgsConstructor
11 @Builder
12 public class Category {
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @Column(nullable = false, unique = true)
18     private String name;
19
20     private String description;
21
22     @OneToMany(mappedBy = "category",
23               cascade = CascadeType.ALL,
24               fetch = FetchType.LAZY)
25     private List<Product> products;
26 }
```

model/Product.java

```
1  package com.smartinventory.model;
2
3  import jakarta.persistence.*;
4  import lombok.*;
5  import org.hibernate.annotations.*;
6  import java.math.BigDecimal;
7  import java.time.LocalDateTime;
8
9  @Entity
10 @Table(name = "products")
11 @Getter @Setter
12 @NoArgsConstructor @AllArgsConstructor
13 @Builder
14 public class Product {
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @Column(nullable = false)
20     private String name;
21
22     private String description;
23
24     @Column(nullable = false, precision = 10, scale = 2)
25     private BigDecimal price;
26
27     @Column(name = "stock_quantity", nullable = false)
28     private Integer stockQuantity;
29
30     @Column(name = "image_url")
```

```

31     private String imageUrl;
32
33     @ManyToOne(fetch = FetchType.LAZY)
34     @JoinColumn(name = "category_id", nullable = false)
35     private Category category;
36
37     @CreationTimestamp
38     @Column(name = "created_at", updatable = false)
39     private LocalDateTime createdAt;
40
41     @UpdateTimestamp
42     @Column(name = "updated_at")
43     private LocalDateTime updatedAt;
44 }

```

Why?

Why BigDecimal? Double uses floating-point: $0.1 + 0.2 = 0.30000000000000004$. For financial calculations, BigDecimal provides exact precision.

Why FetchType.LAZY? With EAGER, loading 100 products loads 100 categories immediately. LAZY loads only when accessed.

8.3 Solution 1.3 — Repositories

repository/ProductRepository.java

```

1  package com.smartinventory.repository;
2
3  import com.smartinventory.model.Product;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import java.util.List;
6
7  public interface ProductRepository
8      extends JpaRepository<Product, Long> {
9      List<Product> findByCategoryId(Long categoryId);
10 }

```

8.4 Solution 1.4 — DTOs

dto/response/ProductResponse.java

```

1  package com.smartinventory.dto.response;
2
3  import java.math.BigDecimal;
4

```

```
5 public record ProductResponse(  
6     Long id,  
7     String name,  
8     String description,  
9     BigDecimal price,  
10    Integer stockQuantity,  
11    String imageUrl,  
12    String categoryName  
13 ) {}
```

dto/request/ProductRequest.java

```
1 package com.smartinventory.dto.request;  
2  
3 import java.math.BigDecimal;  
4  
5 public record ProductRequest(  
6     String name,  
7     String description,  
8     BigDecimal price,  
9     Integer stockQuantity,  
10    String imageUrl,  
11    Long categoryId  
12 ) {}
```

8.5 Solution 1.5 — Service Layer

exception/ResourceNotFoundException.java

```
1 package com.smartinventory.exception;  
2  
3 public class ResourceNotFoundException  
4     extends RuntimeException {  
5     public ResourceNotFoundException(String message) {  
6         super(message);  
7     }  
8 }
```

service/impl/ProductServiceImpl.java

```
1 package com.smartinventory.service.impl;  
2  
3 import com.smartinventory.dto.request.ProductRequest;  
4 import com.smartinventory.dto.response.ProductResponse;  
5 import com.smartinventory.exception.*;  
6 import com.smartinventory.model.*;  
7 import com.smartinventory.repository.*;  
8 import com.smartinventory.service.ProductService;
```

```
9 import lombok.RequiredArgsConstructor;
10 import org.springframework.stereotype.Service;
11 import java.util.List;
12 import java.util.stream.Collectors;
13
14 @Service
15 @RequiredArgsConstructor
16 public class ProductServiceImpl
17     implements ProductService {
18
19     private final ProductRepository productRepository;
20     private final CategoryRepository categoryRepository;
21
22     @Override
23     public List<ProductResponse> getAllProducts() {
24         return productRepository.findAll()
25             .stream()
26             .map(this::mapToResponse)
27             .collect(Collectors.toList());
28     }
29
30     @Override
31     public ProductResponse getProductById(Long id) {
32         Product product = productRepository.findById(id)
33             .orElseThrow(() ->
34                 new ResourceNotFoundException(
35                     "Product not found with id: " + id));
36         return mapToResponse(product);
37     }
38
39     @Override
40     public ProductResponse createProduct(
41         ProductRequest request) {
42         Category category = categoryRepository
43             .findById(request.categoryId())
44             .orElseThrow(() ->
45                 new ResourceNotFoundException(
46                     "Category not found with id: "
47                     + request.categoryId()));
48
49         Product product = Product.builder()
50             .name(request.name())
51             .description(request.description())
52             .price(request.price())
53             .stockQuantity(request.stockQuantity())
54             .imageUrl(request.imageUrl())
55             .category(category)
56             .build();
57
58         Product saved = productRepository.save(product);
59         return mapToResponse(saved);
60     }
61 }
```

```

60     }
61
62     private ProductResponse mapToResponse(Product p) {
63         return new ProductResponse(
64             p.getId(), p.getName(),
65             p.getDescription(), p.getPrice(),
66             p.getStockQuantity(), p.getImageUrl(),
67             p.getCategory().getName());
68     }
69 }

```

8.6 Solution 1.6 — Controller

controller/ProductController.java

```

1  package com.smartinventory.controller;
2
3  import com.smartinventory.dto.request.ProductRequest;
4  import com.smartinventory.dto.response.ProductResponse;
5  import com.smartinventory.service.ProductService;
6  import lombok.RequiredArgsConstructor;
7  import org.springframework.http.*;
8  import org.springframework.web.bind.annotation.*;
9  import java.util.List;
10
11  @RestController
12  @RequestMapping("/api/v1/products")
13  @RequiredArgsConstructor
14  public class ProductController {
15
16      private final ProductService productService;
17
18      @GetMapping
19      public ResponseEntity<List<ProductResponse>>
20          getAllProducts() {
21          return ResponseEntity.ok(
22              productService.getAllProducts());
23      }
24
25      @GetMapping("/{id}")
26      public ResponseEntity<ProductResponse>
27          getProductById(@PathVariable Long id) {
28          return ResponseEntity.ok(
29              productService.getProductById(id));
30      }
31
32      @PostMapping
33      public ResponseEntity<ProductResponse>
34          createProduct(

```

```

35         @RequestBody ProductRequest request) {
36         ProductResponse response =
37             productService.createProduct(request);
38         return ResponseEntity
39             .status(HttpStatus.CREATED)
40             .body(response);
41     }
42 }

```

8.7 Solution 1.7 — CORS

config/CorsConfig.java

```

1  package com.smartinventory.config;
2
3  import org.springframework.context.annotation.*;
4  import org.springframework.web.servlet.config.annotation.*;
5
6  @Configuration
7  public class CorsConfig
8      implements WebMvcConfigurer {
9
10     @Override
11     public void addCorsMappings(CorsRegistry registry) {
12         registry.addMapping("/api/**")
13             .allowedOrigins("http://localhost:4200")
14             .allowedMethods("GET", "POST",
15                 "PUT", "DELETE", "OPTIONS")
16             .allowedHeaders("*")
17             .allowCredentials(true)
18             .maxAge(3600);
19     }
20 }

```

8.8 Solution 1.8–1.12 — Angular Setup

src/environments/environment.development.ts

```

1  export const environment = {
2      production: false,
3      apiUrl: 'http://localhost:8080/api/v1'
4  };

```

src/app/app.config.ts

```

1  import { ApplicationConfig } from '@angular/core';

```

```

2 import { provideRouter } from '@angular/router';
3 import { provideHttpClient, withFetch }
4   from '@angular/common/http';
5 import { routes } from '../app.routes';
6
7 export const appConfig: ApplicationConfig = {
8   providers: [
9     provideRouter(routes),
10    provideHttpClient(withFetch())
11  ]
12 };

```

features/products/models/product.model.ts

```

1 export interface Product {
2   id: number;
3   name: string;
4   description: string;
5   price: number;
6   stockQuantity: number;
7   imageUrl: string;
8   categoryName: string;
9 }
10
11 export interface ProductRequest {
12   name: string;
13   description: string;
14   price: number;
15   stockQuantity: number;
16   imageUrl: string;
17   categoryId: number;
18 }

```

features/products/services/product.service.ts

```

1 import { Injectable } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable } from 'rxjs';
4 import { Product, ProductRequest }
5   from '../models/product.model';
6 import { environment }
7   from '../../environments/environment';
8
9 @Injectable({ providedIn: 'root' })
10 export class ProductService {
11   private readonly apiUrl =
12     `${environment.apiUrl}/products`;
13
14   constructor(private http: HttpClient) {}
15
16   getAll(): Observable<Product[]> {

```

```

17     return this.http.get<Product []>(this.apiUrl);
18   }
19
20   getById(id: number): Observable<Product> {
21     return this.http.get<Product>(
22       `${this.apiUrl}/${id}`);
23   }
24
25   create(req: ProductRequest): Observable<Product> {
26     return this.http.post<Product>(this.apiUrl, req);
27   }
28 }

```

features/products/product-list/product-list.component.ts

```

1  import { Component, OnInit } from '@angular/core';
2  import { CommonModule } from '@angular/common';
3  import { ProductService }
4    from '../services/product.service';
5  import { Product } from '../models/product.model';
6
7  @Component({
8    selector: 'app-product-list',
9    standalone: true,
10   imports: [CommonModule],
11   templateUrl: './product-list.component.html'
12 })
13 export class ProductListComponent implements OnInit {
14   products: Product[] = [];
15   loading = true;
16   error = '';
17
18   constructor(private productService: ProductService) {}
19
20   ngOnInit(): void {
21     this.productService.getAll().subscribe({
22       next: (data) => {
23         this.products = data;
24         this.loading = false;
25       },
26       error: (err) => {
27         this.error = 'Failed to load products';
28         this.loading = false;
29       }
30     });
31   }
32 }

```

product-list.component.html

```
<div class="product-list-container">
```

```
<h1>Product Catalog</h1>

@if (loading) {
  <p>Loading products...</p>
}

@if (!loading && !error) {
  <div class="product-grid">
    @for (product of products; track product.id) {
      <div class="product-card">
        <h3>{{ product.name }}</h3>
        <p>{{ product.categoryName }}</p>
        <p>{{ product.price | currency:'USD' }}</p>
        @if (product.stockQuantity == 0) {
          <p class="out-of-stock">Out of Stock</p>
        } @else {
          <p>In Stock: {{ product.stockQuantity }}</p>
        }
      </div>
    } @empty {
      <p>No products found.</p>
    }
  </div>
}
</div>
```

Chapter 9

Solution — Phase 2

9.1 Solution 2.1 — Bean Validation

dto/request/ProductRequest.java (updated)

```
1 package com.smartinventory.dto.request;
2
3 import jakarta.validation.constraints.*;
4 import java.math.BigDecimal;
5
6 public record ProductRequest(
7     @NotBlank(message = "Product name is required")
8     @Size(max = 100,
9         message = "Name must be at most 100 chars")
10    String name,
11
12    @Size(max = 500)
13    String description,
14
15    @NotNull(message = "Price is required")
16    @DecimalMin(value = "0.01",
17        message = "Price must be > 0")
18    BigDecimal price,
19
20    @NotNull(message = "Stock quantity is required")
21    @Min(value = 0,
22        message = "Stock cannot be negative")
23    Integer stockQuantity,
24
25    String imageUrl,
26
27    @NotNull(message = "Category is required")
28    Long categoryId
29 ) {}
```

9.2 Solution 2.2 — Global Error Handler

exception/GlobalExceptionHandler.java

```

1 package com.smartinventory.exception;
2
3 import com.smartinventory.dto.response.ErrorResponse;
4 import org.springframework.http.HttpStatus;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.validation.FieldError;
7 import
    org.springframework.web.bind.MethodArgumentNotValidException;
8 import org.springframework.web.bind.annotation.*;
9 import java.time.LocalDateTime;
10 import java.util.*;
11
12 @RestControllerAdvice
13 public class GlobalExceptionHandler {
14
15     @ExceptionHandler(
16         MethodArgumentNotValidException.class)
17     public ResponseEntity<ErrorResponse>
18         handleValidation(
19         MethodArgumentNotValidException ex) {
20         Map<String, String> errors = new HashMap<>();
21         ex.getBindingResult().getFieldErrors()
22             .forEach((FieldError fe) ->
23             errors.put(fe.getField(),
24                 fe.getDefaultMessage()));
25
26         ErrorResponse response = new ErrorResponse(
27             LocalDateTime.now(),
28             HttpStatus.BAD_REQUEST.value(),
29             "Validation Failed", errors);
30
31         return ResponseEntity
32             .status(HttpStatus.BAD_REQUEST)
33             .body(response);
34     }
35
36     @ExceptionHandler(
37         ResourceNotFoundException.class)
38     public ResponseEntity<ErrorResponse>
39         handleNotFound(
40         ResourceNotFoundException ex) {
41         ErrorResponse response = new ErrorResponse(
42             LocalDateTime.now(),
43             HttpStatus.NOT_FOUND.value(),
44             ex.getMessage(), null);
45         return ResponseEntity
46             .status(HttpStatus.NOT_FOUND)

```

```

47         .body(response);
48     }
49
50     @ExceptionHandler(Exception.class)
51     public ResponseEntity<ErrorResponse>
52         handleGeneric(Exception ex) {
53         ex.printStackTrace();
54         ErrorResponse response = new ErrorResponse(
55             LocalDateTime.now(),
56             HttpStatus.INTERNAL_SERVER_ERROR.value(),
57             "An unexpected error occurred", null);
58         return ResponseEntity
59             .status(HttpStatus.INTERNAL_SERVER_ERROR)
60             .body(response);
61     }
62 }

```

9.3 Solution 2.3–2.5 — Angular Forms

```

                                product-form.component.ts
1  import { Component } from '@angular/core';
2  import { CommonModule } from '@angular/common';
3  import { ReactiveFormsModule, FormGroup,
4      FormControl, Validators } from '@angular/forms';
5  import { Router } from '@angular/router';
6  import { ProductService }
7      from '../services/product.service';
8  import { HttpErrorResponse }
9      from '@angular/common/http';
10 import { priceFormatValidator }
11     from '../validators/price.validator';
12
13 @Component({
14     selector: 'app-product-form',
15     standalone: true,
16     imports: [CommonModule, ReactiveFormsModule],
17     templateUrl: './product-form.component.html'
18 })
19 export class ProductFormComponent {
20     serverErrors: { [key: string]: string } = {};
21     submitting = false;
22
23     form = new FormGroup({
24         name: new FormControl('', [
25             Validators.required,
26             Validators.minLength(2),
27             Validators.maxLength(100)
28         ]),

```

```

29     description: new FormControl('',
30         [Validators.maxLength(500)]),
31     price: new FormControl<number | null>(null, [
32         Validators.required,
33         Validators.min(0.01),
34         priceFormatValidator
35     ]),
36     stockQuantity: new FormControl<number | null>(
37         null, [Validators.required, Validators.min(0)]),
38     imageUrl: new FormControl(''),
39     categoryId: new FormControl<number | null>(
40         null, [Validators.required])
41 });
42
43 constructor(
44     private productService: ProductService,
45     private router: Router
46 ) {
47     this.form.valueChanges.subscribe(() => {
48         this.serverErrors = {};
49     });
50 }
51
52 onSubmit(): void {
53     this.form.markAllAsTouched();
54     if (this.form.invalid) return;
55
56     this.submitting = true;
57     this.productService.create(
58         this.form.value as any
59     ).subscribe({
60         next: () =>
61             this.router.navigate(['/products']),
62         error: (err: HttpResponse) => {
63             this.submitting = false;
64             if (err.status === 400 && err.error?.errors) {
65                 this.serverErrors = err.error.errors;
66             }
67         }
68     });
69 }
70
71 hasError(field: string, type: string): boolean {
72     const ctrl = this.form.get(field);
73     return !!((ctrl?.hasError(type) && ctrl?.touched);
74 }
75 }

```

validators/price.validator.ts

```

1 import { AbstractControl, ValidationErrors }

```

```
2   from '@angular/forms';
3
4   export function priceFormatValidator(
5       control: AbstractControl
6   ): ValidationErrors | null {
7       const value = control.value;
8       if (!value) return null;
9
10      const str = value.toString();
11      const dotIndex = str.indexOf('.');
12      if (dotIndex !== -1) {
13          const decimals = str.length - dotIndex - 1;
14          if (decimals > 2) return { invalidPrice: true };
15      }
16      return null;
17  }
```

Chapter 10

Solution — Phase 3

10.1 Solution 3.1–3.2 — Pagination & Specifications

repository/ProductSpecification.java

```
1 package com.smartinventory.repository;
2
3 import com.smartinventory.model.Product;
4 import org.springframework.data.jpa.domain.Specification;
5 import java.math.BigDecimal;
6
7 public class ProductSpecification {
8
9     public static Specification<Product>
10         hasName(String name) {
11         return (root, query, cb) -> {
12             if (name == null || name.isBlank())
13                 return null;
14             return cb.like(
15                 cb.lower(root.get("name")),
16                 "%" + name.toLowerCase() + "%");
17         };
18     }
19
20     public static Specification<Product>
21         hasCategory(Long categoryId) {
22         return (root, query, cb) -> {
23             if (categoryId == null) return null;
24             return cb.equal(
25                 root.get("category").get("id"),
26                 categoryId);
27         };
28     }
29
30     public static Specification<Product>
31         hasPriceRange(
32             BigDecimal min, BigDecimal max) {
33         return (root, query, cb) -> {
```

```

34         if (min == null && max == null)
35             return null;
36         if (min != null && max != null)
37             return cb.between(
38                 root.get("price"), min, max);
39         if (min != null)
40             return cb.greaterThanOrEqualTo(
41                 root.get("price"), min);
42         return cb.lessThanOrEqualTo(
43             root.get("price"), max);
44     };
45 }
46 }

```

10.2 Solution 3.3–3.4 — Signals & RxJS

product-list.component.ts (with Signals & RxJS)

```

1  import { Component, OnInit, OnDestroy,
2      signal, computed } from '@angular/core';
3  import { CommonModule } from '@angular/common';
4  import { ReactiveFormsModule, FormControl }
5      from '@angular/forms';
6  import { Subject, takeUntil } from 'rxjs';
7  import { debounceTime, distinctUntilChanged,
8      switchMap, tap } from 'rxjs/operators';
9  import { ProductService }
10     from '../services/product.service';
11  import { Product, PageResponse }
12     from '../models/product.model';
13
14  @Component({
15     selector: 'app-product-list',
16     standalone: true,
17     imports: [CommonModule, ReactiveFormsModule],
18     templateUrl: './product-list.component.html'
19 })
20  export class ProductListComponent
21     implements OnInit, OnDestroy {
22
23     searchControl = new FormControl('');
24
25     products = signal<Product[]>([]);
26     loading = signal(true);
27     currentPage = signal(0);
28     totalPages = signal(0);
29     pageSize = signal(10);
30
31     isEmpty = computed(() =>

```

```
32     this.products().length === 0
33     && !this.loading());
34
35     private destroy$ = new Subject<void>();
36
37     constructor(
38         private productService: ProductService) {}
39
40     ngOnInit(): void {
41         this.loadProducts('');
42
43         this.searchControl.valueChanges.pipe(
44             debounceTime(300),
45             distinctUntilChanged(),
46             tap(() => {
47                 this.loading.set(true);
48                 this.currentPage.set(0);
49             }),
50             switchMap(term =>
51                 this.productService.search(
52                     term || '', 0, this.pageSize()))
53             .takeUntil(this.destroy$)
54             ).subscribe({
55                 next: (res: PageResponse<Product>) => {
56                     this.products.set(res.content);
57                     this.totalPages.set(res.totalPages);
58                     this.loading.set(false);
59                 },
60                 error: () => this.loading.set(false)
61             });
62     }
63
64     loadProducts(search: string): void {
65         this.loading.set(true);
66         this.productService.search(
67             search, this.currentPage(), this.pageSize())
68             .pipe(takeUntil(this.destroy$))
69             .subscribe({
70                 next: (res) => {
71                     this.products.set(res.content);
72                     this.totalPages.set(res.totalPages);
73                     this.loading.set(false);
74                 },
75                 error: () => this.loading.set(false)
76             });
77     }
78
79     ngOnDestroy(): void {
80         this.destroy$.next();
81         this.destroy$.complete();
82     }
```

83 }

Chapter 11

Solution — Phase 4

11.1 Solution 4.1 — User Entity

model/User.java (implements UserDetails)

```
1 package com.smartinventory.model;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import org.hibernate.annotations.CreationTimestamp;
6 import org.springframework.security.core.*;
7 import org.springframework.security.core.authority.*;
8 import org.springframework.security.core.userdetails.*;
9 import java.time.LocalDateTime;
10 import java.util.*;
11
12 @Entity @Table(name = "users")
13 @Getter @Setter @NoArgsConstructor
14 @AllArgsConstructor @Builder
15 public class User implements UserDetails {
16     @Id @GeneratedValue(strategy =
17         GenerationType.IDENTITY)
18     private Long id;
19
20     @Column(name = "first_name", nullable = false)
21     private String firstName;
22
23     @Column(name = "last_name", nullable = false)
24     private String lastName;
25
26     @Column(nullable = false, unique = true)
27     private String email;
28
29     @Column(nullable = false)
30     private String password;
31
32     @Enumerated(EnumType.STRING)
33     @Column(nullable = false)
```

```

34     private Role role;
35
36     @CreationTimestamp
37     @Column(name = "created_at", updatable = false)
38     private LocalDateTime createdAt;
39
40     @Override
41     public Collection<? extends GrantedAuthority>
42         getAuthorities() {
43         return List.of(new SimpleGrantedAuthority(
44             "ROLE_" + role.name()));
45     }
46
47     @Override
48     public String getUsername() { return email; }
49     @Override
50     public boolean isAccountNonExpired() {
51         return true; }
52     @Override
53     public boolean isAccountNonLocked() {
54         return true; }
55     @Override
56     public boolean isCredentialsNonExpired() {
57         return true; }
58     @Override
59     public boolean isEnabled() { return true; }
60 }

```

11.2 Solution 4.2–4.3 — JWT & Security Config

See the complete solutions in the Markdown version of this TP. The key patterns are:

1. `JwtService`: Uses `io.jsonwebtoken` to generate/validate tokens
2. `JwtAuthenticationFilter`: Extends `OncePerRequestFilter`, extracts Bearer token, validates, sets `SecurityContextHolder`
3. `SecurityConfig`: Disables CSRF, sets STATELESS sessions, permits `/auth/**`, requires ADMIN for `/admin/**`

11.3 Solution 4.5–4.7 — Angular Auth

core/interceptors/auth.interceptor.ts

```

1 import { HttpInterceptorFn }
2   from '@angular/common/http';
3 import { inject } from '@angular/core';
4 import { StorageService }

```

```
5   from '../services/storage.service';
6
7   export const authInterceptor: HttpInterceptorFn =
8     (req, next) => {
9       if (req.url.includes('/auth/'))
10         return next(req);
11
12       const storage = inject(StorageService);
13       const token = storage.getToken();
14
15       if (token) {
16         const cloned = req.clone({
17           headers: {
18             Authorization: `Bearer ${token}`
19           }
20         });
21         return next(cloned);
22       }
23
24       return next(req);
25     };
```

core/guards/auth.guard.ts

```
1   import { CanActivateFn, Router }
2     from '@angular/router';
3   import { inject } from '@angular/core';
4   import { AuthService }
5     from '../services/auth.service';
6
7   export const authGuard: CanActivateFn = () => {
8     const auth = inject(AuthService);
9     const router = inject(Router);
10
11     if (auth.isLoggedIn()) return true;
12     return router.createUrlTree(['/auth/login']);
13   };
```

Chapter 12

Solution — Phase 5

12.1 Solution 5.1 — N+1 Fix

ProductRepository with @EntityGraph

```
1 @EntityGraph(attributePaths = {"category"})
2 Page<Product> findAll(
3     Specification<Product> spec, Pageable pageable);
4
5 @EntityGraph(attributePaths = {"category"})
6 Page<Product> findAll(Pageable pageable);
```

Why?

Before: 1 query for products + N queries for categories = N+1 queries

After: 1 query with LEFT JOIN = 1 query total

12.2 Solution 5.3 — Docker

docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: smart_inventory
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL",
            "pg_isready -U postgres"]
```

```
    interval: 10s
    timeout: 5s
    retries: 5

api:
  build: ./smart-inventory-api
  environment:
    SPRING_DATASOURCE_URL:
      jdbc:postgresql://db:5432/smart_inventory
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
  ports:
    - "8080:8080"
  depends_on:
    db:
      condition: service_healthy

ui:
  build: ./smart-inventory-ui
  ports:
    - "80:80"
  depends_on:
    - api

volumes:
  postgres_data:
```

Chapter 13

Weekly Time Distribution

13.1 Daily Schedule (2 hours/day)

Day	Focus	Activities
1	 + 	30min: Read docs/video. 90min: Implement API endpoint
2		30min: Read docs. 90min: Complete backend, validation, errors
3	 + 	30min: Read Angular docs. 90min: Build service & component
4		30min: Read docs. 90min: Complete UI, connect to backend
5		Full 2h: Refactoring, debugging (Network Tab, SQL logs)
6		Full 2h: Add a feature using only official docs & error messages

Chapter 14

Essential Resources

Resource	Link
Spring Boot Reference	https://docs.spring.io/spring-boot/reference/
Spring Data JPA	https://docs.spring.io/spring-data/jpa/reference/
Angular Documenta- tion	https://angular.dev
RxJS Documentation	https://rxjs.dev
JWT Introduction	https://jwt.io/introduction
Docker Compose	https://docs.docker.com/compose/

End of TP — Smart Inventory & Order Platform

© 2026 Harvard University — School of Engineering and Applied Sciences